МИРЭА
Российский технологический университет

Создание процессов и исполнение программ

Мухортова Н.Н.

Цель

Этот раздел объясняет, что такое процесс и чем он отличается от программы.

Вы изучите взаимодействие между родительским процессом и одним или более порожденными процессами (подпроцессами). Также обсуждается создание подпроцесса, исполнение другой программы в том же процессе, заверение процесса и ожидание завершения порожденного процесса

Процесс

Процесс представляет собой исполняющуюся программу вместе с необходимым ей окружением. Окружение процесса состоит из:

. информации о процессе, содержащейся в различных системных структурах данных

. содержимого регистров процессора (контекста процесса)

. пользовательского стека процесса и системного стека, используемого при обработке системных вызовов

. пользовательской области, которая содержит информацию об открытых файлах, текущей директории, обработке сигналов и т.д.

Процесс

Образ процесса есть размещение процесса в памяти в заданный момент времени. Образ процесса представляет собой набор отображённых в виртуальную память сегментов, представляющих собой код основной программы, сегмент данных, сегменты кода и данных разделяемых библиотек, отображённые на память файлы, сегменты разделяемой памяти System V IPC и др. Обратите внимание, что программа является частью образа процесса.

Благодаря тому, что отображённые в память сегменты не обязаны быть загружены полностью и подкачиваются с диска по мере необходимости, а также благодаря тому, что сегменты кода и неизмененные части сегментов данных у различных процессов могут размещаться в разделяемой памяти, общий объем образов всех процессов в системе может превосходить объем физического ОЗУ.

Процесс

Каждый процесс имеет собственное виртуальное адресное пространство и, таким образом, защищён от ошибок и злонамеренных действий кода, запущенного другими пользователями.

В современных системах семейства Unix, в рамках процесса может быть создано несколько нитей или потоков исполнения. В рамках данного раздела мы не изучаем многопоточное исполнение и рассматриваем только традиционные процессы с единственной нитью.

Процесс представляет собой изменяющийся со временем динамический объект. Процесс может создать один или более порождённых процессов, используя системный вызов fork(2). Кроме того, он может изменить свою программу, используя системный вызов exec(2). Процесс может приостановить исполнение, используя системные вызовы wait(2) или waitid(2). Он может также завершить своё исполнение системным вызовом exit(2).

Многие прикладные программы реализованы не в виде единой монолитной программы, а в виде нескольких или даже множества взаимодействующих процессов. Это позволяет

повысить отказоустойчивость: аварийное завершение одного процесса не обязательно приводит к нарушению работы всей программы,

реализовать принцип минимума привилегий: каждый процесс исполняется с теми правами, которые необходимы ему для выполнения его функций, но не более,

обойти ограничение на размер образа процесса, что было очень актуально на 16-разрядных компьютерах, а в последние годы становится актуально на 32-разрядных машинах, а также некоторые другие ограничения, например, ограничение на количество одновременно открытых файлов,

распределить вычислительную нагрузку по нескольким процессорным ядрам или виртуальным процессорам. Эту задачу можно также решать с использованием многопоточности, но многие современные прикладные программы были разработаны до того, как были стандартизованы API и средства для многопоточного программирования.

Создание процесса

Системный вызов fork(2) создаёт новый процесс, исполняющий копию исходного процесса. В основном, новый процесс (порождённый или дочерний) идентичен исходному (родителю). В описании fork(2) перечислены атрибуты, которые порождённый процесс наследует от родителя, и различия между ними.

Дочерний процесс наследует все отображённые на память файлы и вообще все сегменты адресного пространства, все открытые файлы, идентификаторы группы процессов и сессии, реальный и эффективный идентификаторы пользователя и группы, ограничения rlimit, текущий каталог, а также ряд других параметров, которые будут обсуждаться в следующих разделах.

Создание процесса

Дочерний процесс НЕ наследует: идентификатор процесса, идентификатор родительского процесса, а также захваченные участки файлов. В большинстве Unix-систем, дочерний процесс не наследует нити исполнения, кроме той, из которой был вызван fork(2). Однако в Solaris предоставляется системный вызов forkall(2), который воспроизводит в дочернем процессе все нити родительского. Этот системный вызов не имеет аналогов в стандарте POSIX и его использование приведёт к проблемам при переносе вашей программы на другие платформы.

Создание процесса

После возврата из fork(2), оба процесса продолжают исполнение с той точки, где fork(2) был вызван. Процессы могут узнать, кто из них является родителем, а кто порождённым, на основании значения, возвращённого fork(2).

Родительский процесс получает идентификатор порождённого процесса, положительное число. Порождённый процесс получает нулевое значение. Как правило, за fork(2) следует оператор if или switch, который определяет, какой код исполнять для родительского и какой для порождённого процесса.

Создание процесса

Системный вызов fork(2) может завершиться неудачей, если вы пытаетесь превысить разрешённое количество процессов для каждого пользователя или общее количество процессов в системе. Эти два ограничения устанавливаются при конфигурации операционной системы. Если fork(2) завершается неуспехом, он возвращает значение -1. Рекомендуется проверять код возврата этого и остальных системные вызовы на предмет неудачного завершения.

Системный вызов fork(2)

Эта иллюстрация показывает родительский процесс до вызова fork(2) и после того, как этот вызов возвратил управление. После fork(2) исполняются два процесса с различными идентификаторами. Сегменты текста, данных и стека у родительского и порождённого процессов идентичны. Для программы с разделяемым сегментом TEXT (компилятор по умолчанию создаёт именно такие программы), сегмент кода, в действительности, будет одним и тем же физическим сегментом. После fork(2) только увеличится счётчик ссылок на него.

Системный вызов fork(2)

Оба процесса имеют почти одинаковые пользовательские области. Так как пользовательская область содержит таблицу дескрипторов файлов, все перенаправления ввода/вывода, сделанные в родительском процессе, наследуются потомком. Захваты файлов являются собственностью процесса, поэтому подпроцесс не унаследует захватов файлов, сделанных родителем. Отображение файлов на память при fork(2) сохраняется.

Системный вызов fork(2)

При этом, если сегмент отображался в режиме MAP_SHARED, родительский и порождённый процессы используют одну и ту же физическую память; если такой сегмент доступен для модификации, то родитель и потомок будут видеть вносимые в эту память изменения. Большинство разделяемых сегментов в Unix-системах — это сегменты кода, недоступные для модификации.

Если же сегмент отображался как MAP_PRIVATE или MAP_ANON (по умолчанию, сегменты данных и стека отображаются именно так), процессы получают собственные копии соответствующих страниц. В действительности, копирование происходит при первой записи в страницу. Непосредственно после fork(2), приватные сегменты остаются разделяемыми, но система устанавливает на страницы этих сегментов защиту от записи.

Чтение таких страниц происходит без изменений, но при попытке модификации такой страницы, диспетчер памяти генерирует исключение защиты памяти. Ядро перехватывает это исключение, но вместо завершения процесса по SIGSEGV (как это происходит при обычных ошибках защиты памяти), создаёт копию приватной страницы и отображает эту копию в адресное пространство того процесса, который пытался произвести запись. Такое поведение называется копированием при записи (copy-on-write)

Системный вызов fork(2)

Поскольку большая часть памяти родительского и порождённого процессов является разделяемой, fork(2) представляет собой относительно дешёвую операцию, во всяком случае, существенно более дешёвую, чем создание процесса в Win32, но существенно более дорогую, чем создание нити в рамках процесса

Системный вызов fork(2) - Пример

Этот пример иллюстрирует создание подпроцесса. После fork(2) два процесса будут исполнять одну и ту же программу. Они оба распечатают свой идентификатор процесса и идентификатор родителя. Эта программа работает следующим образом:

8-9 Процесс распечатывает идентификатор своего родительского процесса; при работе в терминальной сессии это обычно идентификатор процесса командного процессора.

11 Создается новый процесс. Новый (порожденный) процесс является [почти] точной копией вызывающего процесса (родителя).

13-14 Оба процесса исполняют этот оператор.

Системный вызов fork(2) - Пример

Файл: fork0.c Замечание: Вызов getppid(2), исполняемый вновь порожденным процессом, может возвратить идентификатор процесса 1, то есть процесса init. После fork(2) родительский и порождённый процесс исполняются независимо. Родитель может завершиться раньше, чем порождённый, особенно если он выполняет меньше действий. Если родительский процесс завершается раньше порождённого, то последний "усыновляется" процессом init, то есть система устанавливает процесс с идентификатором 1 в качестве родителя подпроцесса.

```
5 main()
6 {
7
8       printf("[%ld] parent process id: %ld\n",
9       getpid(), getppid());
10
11       fork();
12
13       printf("\n\t[%ld] parent process id: %ld\n",
14       getpid(), getppid());
15 }
```

```
$ fork0
[18607] parent process id: 18606

        [18608] parent process id: 18607

        [18607] parent process id: 18606
```

Системный вызов fork(2) - Пример 2

Эта программа создает два процесса: родитель распечатывает заглавные буквы, а порожденный - строчные.

13-16 Значение, возвращенное fork(2), присваивается переменной pid. Положительное значение pid означает родительский процесс.

17-20 Нулевое значение pid означает порожденный процесс.

21-24 Если значение, возвращаемое fork(2), равно -1, то произошла ошибка. В этом случае вызывается функция perror(3), которая распечатывает сообщение о причине неуспеха. Затем программа завершается.

21-24 Как родительский, так и порожденный процессы распечатывают буквы этим оператором for. Внутренний цикл здесь просто потребляет время процессора, чтобы происходило переключение процессов. Это приводит к тому, что большие и маленькие буквы на выводе перемешиваются. Иначе оба процесса быстро бы завершались в течении одного кванта времени.

Системный вызов fork(2) - Пример 2

Оба процесса имеют одинаковую возможность получить управление. Поэтому любой из них может начать исполнение первым.

```
 7 main(int argc, char *argv[ ])   /* demonstrate fork(2) */
 8 {
 9    char ch, first, last;
10        pid_t pid;
11    int i;
12
13    if ((pid = fork()) > 0) {   /* parent */
14                first = 'A';
15        last = 'Z';
16    }
17        else if (pid == 0) {          /* child */
18        first = 'a';
19        last = 'z';
20    }
21        else {                  /* cannot fork(2) */
22        perror(argv[0]);
23        exit(1);
24    }
25        for (ch = first; ch <= last; ch++) {
26        /* delay loop */
27        for (i = 0; i < Bignumber; i++)
28            ;   /* null */
29        write(1, &ch, 1);
30    }
31
32    exit(0);
33 }
```

Наблюдая за выводом программы-примера, можно заметить следующие факты:

. каждый из процессов выводит свой текст в правильном порядке, то есть, как заглавные, так и строчные буквы идут в алфавитном порядке.

. время исполнения каждого процесса непредсказуемо.

. невозможно предсказать, какой из процессов закончится первым.

Процесс

Как правило, существует несколько процессов, поочерёдно использующих центральный процессор. Каждому процессу выделяется определённое количество времени процессора (квант). Когда процесс израсходовал свой квант, процессор может быть передан другому процессу. Этот механизм предотвращает захваты процессора одним процессом.

Процесс

Обратите внимание, что при первых двух вызовах приглашение shell появилось в середине строки вывода. Это случилось, потому что родительский процесс завершился раньше порождённого. shell выдает приглашение при завершении родительского процесса, не ожидая завершения его подпроцессов.

СИСТЕМНЫЙ ВЫЗОВ fork(2) - ПРИМЕР (ВЫВОД)

```
$ fork1
abcABdeCDfEFgGhijklHIJKmnopLMNOPQRqrstSTUuvwxyVWXYZ$ z

$ fork1
aAbBCDEFGHIJcdefghijkKLMNOPQRSlmnTUVopqrstWXYZ$ uvwxyz

$ fork1
abABCcdefgDEFGhijklmnoHIJKLMpNqOPrQsRtuvSTUwxVWyXzYZ$

$ fork1
abcAdeBCfDghEFGHIijkJKlLMNOmnopqPQRrsSTtuUVvWwxXyzYZ$
```

Исполнение программы

Процесс может заменить текущую программу на новую, исполнив системный вызов exec(2). Этот вызов заменяет текст, данные, стек и остальные сегменты виртуального адресного пространства текущей программы на соответствующие сегменты новой программы. Однако пользовательская область при этом вызове сохраняется.

Исполнение программы

Существует шесть вариантов системного вызова exec(2). Обратите внимание, что за exec идет одна или несколько букв:

l (список аргументов),

v (вектор аргументов),

e (изменение среды) или

p (использование переменной PATH).

Исполнение программы

Формат вызова exec(2) определяет, какие данные передаются новой программе. Ниже приведены параметры различных версий exec(2): path указывает на строку, которая содержит абсолютное или относительное имя загрузочного модуля. file указывает на строку, содержащую имя загружаемого файла, который находится в одной из директорий, перечисленных в переменной PATH.

Исполнение программы

arg0,...,argn указывают на строки - значения параметров, которые надо передать новой программе. Эти значения помещаются в вектор argv[] - параметр функции main() новой программы. Количество параметров помещается в параметр argc функции main(). Список параметров должен завершаться нулевым указателем.

Исполнение программы

argv[] вектор указателей на строки, содержащие параметры, которые нужно передать новой программе. Преимущество использования argv состоит в том, что список параметров может быть построен динамически. Последний элемент вектора должен содержать нулевой адрес.

Исполнение программы

envp[] вектор указателей на строки, представляющие новые переменные среды для новой программы. Значения элементов этого массива копируются в параметр envp[] функции main() новой программы. Аналогично, environ новой программы указывает на envp[0] новой программы. Последний элемент envp[] должен содержать нулевой адрес.

Исполнение программы

cnup указатель на вектор указателей на строки, представляющие новые переменные среды новой программы; в действительности то же что и envp[].

arg0 или argv[0], следует устанавливать равным последней компоненте path или параметру file, то есть равным имени загружаемой программы.

Пример

Эта программа будет использоваться для демонстрации системных вызовов семейства exec(2) в последующих примерах. Эта программа распечатывает свои аргументы командной строки и переменные среды исполнения

. 12-13 Этот цикл распечатывает аргументы командной строки

. 16-17 Этот цикл распечатывает переменные среды исполнения

```
 1 #include <stdio.h>
 2 extern char **environ;
 3
 4 /* program to be exec(2)'ed */
 5
 6 main(int argc, char *argv[ ])
 7 {
 8     char **p;
 9         int n;
10
11     printf("My input parameters(argv) are:\n");
12         for (n = 0; n < argc; n++)
13         printf("    %2d: '%s'\n", n, argv[n]);
14
15     printf("\nMy environment variables are:\n");
16         for (p = environ; *p != 0; p++)
17         printf("    %s\n", *p);
18 }
```

```
$ newpgm parm1 parm2 parm3
My input parameters(argv) are:
    0: 'newpgm'
    1: 'parm1'
    2: 'parm2'
    3: 'parm3'

My environment variables are:
   HOME=/uxm2/jrs
   LOGNAME=jrs
   MAIL=/var/mail/jrs
   PATH=/usr/bin:/usr/lbin/:/uxm2/jrs/bin:.
   TERM=5420
   TZ=EST5EDT
```

Использование execl(2) - Пример

Эта программа демонстрирует использование системного вызова execl(2)

. 9-10 Системный вызов execl(2) использует список параметров

newpgm имя программы, которую следует исполнить newpgm значение argv[0] новой программы

parm1 значение argv[1] новой программы

parm2 значение argv[2] новой программы

parm3 значение argv[3] новой программы (char *)0 конец списка параметров.

Он необходим, потому что в C/C++ функция с переменным количеством аргументов (a execl(2) является именно такой функцией) не имеет встроенных в язык средств, чтобы определить, сколько ей было передано аргументов (см. varargs(3EXT)).

```
 1 #include <unistd.h>
 2 #include <stdio.h>
 3
 4 main()
 5 {
 6
 7     printf("this is the original program\n");
 8
 9     execl("newpgm", "newpgm", "parm1", "parm2",
10             "parm3", (char *) 0);
11
12     perror("This line should never get printed\n");
13 }
```

```
$ exec1
this is the original program
My input parameters(argv) are:
    0: 'newpgm'
    1: 'parm1'
    2: 'parm2'
    3: 'parm3'

My environment variables are:
   HOME=/uxm2/jrs
   LOGNAME=jrs
   MAIL=/var/mail/jrs
   PATH=/usr/bin:/usr/lbin/:/uxm2/jrs/bin:.
   TERM=5420
   TZ=EST5EDT
```

Завершение процесса

Системный вызов exit(2) предназначен для завершения процесса. Он прекращает исполнение вашей программы. В качестве параметра exit(2) передаётся код завершения в диапазоне от 0 до 255. По соглашению, значение 0 означает, что программа завершилась нормально. Значения от 1 до 255 означают, что программа завершилась из-за какой-либо ошибки

Завершение процесса

Полезно использовать для каждого типа ошибки свой код завершения. Код завершения может быть получен родительским процессом через системный вызов wait(2). Это будет обсуждаться далее. Код завершения команды, исполненной из shell, доступен как переменная shell ${?}. Это полезно при написании командных файлов, выполняющих ветвление в зависимости от кода завершения, возвращённого командой.

Завершение процесса

exit(2) осуществляет действия по очистке, такие как закрытие всех открытых файлов и исполнение деструкторов статических переменных C++. Системный вызов _exit(2) сокращает эти действия по очистке. Например, _exit(2) не очищает буфера стандартной библиотеки ввода-вывода.

Завершение процесса

Вызов _exit(2) необходимо использовать в аварийных ситуациях, например, когда вы подозреваете повреждение памяти вашего процесса и имеете основания предполагать, что сброс буферов стандартной библиотеки может привести к записи в файлы некорректных данных.

Завершение процесса

В языках C/C++, возврат управления из функции main оператором return эквивалентен вызову exit(2). В действительности, такой возврат приводит к вызову exit(2). В этом можно убедиться, просмотрев ассемблерные исходники стартового файла среды исполнения языка C (crt1.o) или пройдя соответствующий оператор в отладчике (отладчик необходимо переключить из режима показа исходного текста в режим показа деассемблированного кода)

Сигналы

Еще одна возможная причина завершения процесса в Unix — это сигналы. Сигнал — это предоставляемое Unix средство обработки ошибок и исключительных ситуаций, иногда используемое для других целей, например для межпроцессной коммуникации. Подробнее сигналы и их обработка будут рассматриваться в разделе «Сигналы».

Сигналы

Сигналы могут возникать: При ошибках программирования: деление на ноль (SIGFPE), ошибки защиты памяти (SIGSEGV), ошибки обращения к памяти (SIGBUS). В ответ на действия пользователя: нажатие некоторых клавиш на терминале приводит к посылке сигналов процессам соответствующей терминальной сессии. В ответ на различные события: разрыв терминальной сессии (SIGHUP), разрыв трубы или сокета (SIGPIPE), завершение операции асинхронного ввода-вывода (настраивается при формировании запроса на ввод-вывод), срабатывание будильника (SIGALRM).

Сигналы

Также сигналы могут программно посылаться одними процессами другим процессам при помощи системных вызовов kill(2) и sigsend(2). Каждый тип сигнала идентифицируется номером. Стандарт POSIX описывает 32 различных сигнала (нумерация начинается с 1); в ОС Solaris предусмотрено 64 типа сигналов. Большинство необработанных сигналов приводит к завершению процесса, получившего этот сигнал. Это считается аварийным завершением процесса и отличается от завершения процесса по exit(2) или _exit(2).

Сигналы

Некоторые сигналы, например, SIGTSTP, SIGTTIN, SIGTTOUT, приводят не к завершению процесса, а к его приостановке. Приостановленный процесс находится в специальном состоянии, которое отличается от ожидания в блокирующемся системном вызове или на примитиве синхронизации. Приостановленный процесс может быть продолжен сигналом SIGCONT. Приостановка процессов используется при реализации управления заданиями, которое рассматриваются в разделе «Терминальный ввод-вывод», и отладчиками, которые в нашем курсе не рассматриваются

Ожидание порожденного процесса

После завершения по exit(2) или по сигналу, процесс переходит в состояние, известное как «зомби». В этом состоянии процесс не исполняется, не имеет пользовательской области, адресного пространства и открытых файлов и не использует большинство других системных ресурсов. Однако «зомби» занимает запись в таблице процессов и сохраняет идентификатор процесса и идентификатор родительского процесса. Эта запись используется для хранения слова состояния процесса, в котором хранится код завершения процесса (параметр exit(2)), если процесс завершился по exit(2) или номер сигнала, если процесс завершился по сигналу.

Ожидание порожденного процесса

На самом деле, главным назначением «зомби» является защита идентификатора процесса (pid) от переиспользования. Дело в том, что родительские процессы идентифицируют своих потомков на основе их pid, а ядро может использовать свободные pid для вновь создаваемых процессов. Поэтому, если бы не существовало записей-«зомби», была бы возможна ситуация, когда потомок с pid=21285 завершается, а родитель, не получив код возврата этого потомка, создает новый подпроцесс и система выделяет ему тот же pid. После этого родитель уже не сможет объяснить системе, какой из потомков с pid-21285 его интересует, и не сможет понять, к какому из потомков относится полученное слово состояния.

Ожидание порожденного процесса

Также, если «зомби» является последним из группы процессов или сессии, то соответствующая группа процессов или сессия продолжают существовать, пока существует зомби. В выводе команды ps(1), процессы-зомби отмечаются надписью в колонке, где для нормальных процессов выводится имя программы. Также, в форматах вывода, в которых показывается статус исполнения процесса, процессы-зомби имеют статус Z.

Ожидание порожденного процесса

Название «зомби» связано с тем, что обычные процессы можно «убить» сигналом, но процессы-зомби на сигналы не реагируют, то есть «убить» их невозможно. Для уничтожения «зомби», родитель такого процесса должен считать его слово состояния системным вызовом wait(2), waitpid(2) или waitid(2). Эти системные вызовы рассматриваются далее. В системах семейства Unix код завершения процесса может быть считан только родителем этого процесса.

Ожидание порожденного процесса

Идентификаторы процессов и записи в таблице процессов представляют собой ценные системные ресурсы, поэтому хорошим тоном считается как можно быстрее освобождать ненужные записи. Если ваша программа создает подпроцессы, она должна позаботиться о своевременном сборе их кодов завершения.

Ожидание порожденного процесса

Если родительский процесс завершается, все его потомки, как продолжающие исполнение, так и «зомби», усыновляются процессом с pid=1. При нормальной работе системы этот процесс исполняет программу /bin/init, которая большую часть времени проводит в системном вызове wait(2), отслеживая состояние запущенных ею системных процессов. Поэтому коды завершения усыновленных «зомби» быстро считываются и соответствующие записи в таблице процессов освобождаются

Ожидание порожденного процесса - wait(2)

Процесс может синхронизоваться с завершением порожденного процесса с помощью системного вызова wait(2). Если вызывается wait(2), а у процесса нет ни одного незавершенного подпроцесса, wait(2) немедленно возвращает -1 и устанавливает errno равной ECHILD.

Ожидание порожденного процесса - wait(2)

Иначе вызывающий процесс:

1. засыпает, если существует незавершившийся подпроцесс.

2. возвращает управление немедленно, если существует подпроцесс, который уже завершился, но к нему не применялся wait(2). В обоих вышеперечисленных случаях, wait(2) возвращает идентификатор завершившегося подпроцесса. Кроме того, слово состояния подпроцесса сохраняется в параметре wait(2).

3. возвращает значение -1 и устанавливает errno в EINTR, если wait(2) был прерван сигналом. Если это произошло, а вы по-прежнему хотите дождаться завершения подпроцесса, вы должны еще раз вызвать wait(2).

Ожидание порожденного процесса - wait(2)

Параметр wait(2) - указатель на целое число, по которому размещается слово состояния подпроцесса. Если вас не интересует слово состояния подпроцесса, вы можете использовать нулевой указатель.

Ожидание порожденного процесса - wait(2)

Подпроцесс может завершиться штатно (вызовом exit(2)), или он может быть убит необработанным сигналом. Эти два варианта можно различить анализом содержимого младших 16 бит слова состояния, которое формирует wait(2). Младший байт слова состояния содержит номер сигнала или ноль, если процесс был завершен по exit(2). Вспомним, что нумерация сигналов начинается с 1, то есть сигнала с номером ноль не существует. Второй по старшинству байт содержит параметр exit(2) или 0, если процесс завершился по сигналу. Содержимое старших двух байтов целочисленного значения не определено, на практике оно обычно 0. В заголовочном файле определено несколько макросов, используемых для анализа содержимого слова состояния. Эти макросы описаны на странице руководства wstat(2) и будут обсуждаться далее

Ожидание порожденного процесса - wait(2)

Если процесс ожидает завершения нескольких подпроцессов, то порядок, в котором они завершатся, неизвестен, а порядок получения слов состояния может не соответствовать порядку их завершения. Поэтому ваша программа не должна зависеть от предполагаемого порядка завершения подпроцессов

Слово состояния wait(2)

Когда процесс ожидает получения слова состояния своих подпроцессов с использованием wait(2) или waitpid(3C), то это слово может быть проанализировано при помощи макросов, определенных в . Эти макросы обсуждаются на странице руководства wstat(5).

Слово состояния wait(2)

WIFEXITED(stat) Ненулевое значение, если это слово состояния получено от подпроцесса, завершившегося по exit(2).

WEXITSTATUS(stat) Если значение WIFEXITED(stat) ненулевое, этот макрос возвращает код завершения, переданный подпроцессом вызову exit(2), или возвращенный его функцией main(), иначе код возврата не определен. WIFSIGNALED(stat) Возвращает ненулевое значение, если это слово состояния получено от подпроцесса, который был принудительно завершен сигналом. WTERMSIG(stat) Если значение WIFSIGNALED(stat) ненулевое, этот макрос возвращает номер сигнала, который вызвал завершение подпроцесса, иначе код возврата не определен

Слово состояния wait(2)

WIFSTOPPED(stat) Возвращает ненулевое значение, если слово состояния получено от приостановленного подпроцесса (wait(2) не реагирует на приостановленные подпроцессы, такое слово состояния может быть получено только вызовом waitpid(2)).

WSTOPSIG(stat) Если значение WIFSTOPPED(stat) ненулевое, этот макрос возвращает номер сигнала, который вызвал приостановку подпроцесса, иначе код возврата не определен

Слово состояния wait(2)

WIFCONTINUED(stat) Возвращает ненулевое значение, если слово состояния получено от процесса, продолжившего исполнение (wait(2) не реагирует на приостановленные подпроцессы, такое слово состояния может быть получено только вызовом waitpid(2)).

WCOREDUMP(stat) Если значение WIFSIGNALED(stat) ненулевое, этот макрос возвратит ненулевое значение, если был создан посмертный дамп памяти (corefайл) завершившегося подпроцесса. Факт создания дампа памяти определяется по номеру сигнала; завершение по некоторым сигналам, таким, как SIGSEGV и SIGFPE, всегда приводит к созданию дампа памяти, завершение по остальным сигналам никогда не создает такой дамп

Ожидание одного процесса - Пример

Эта программа показывает, как ожидать завершения одного подпроцесса, и работает следующим образом: 13-17 Создается подпроцесс, который распечатывает свой идентификатор, идентификатор родительского процесса и код завершения, который будет передан exit(2) в следующем операторе. Родительский процесс запоминает идентификатор подпроцесса в переменной pid.

Ожидание одного процесса - Пример

Замечание: Проверка успешности исполнения fork(2) для краткости опущена, но на практике всегда надо проверять значение, возвращенное системным вызовом.

19 Родительский процесс объявляет, что он ожидает завершения своего подпроцесса.

21 Родитель ожидает завершения порожденного подпроцесса. Может возникнуть одна из двух ситуаций. В одном случае, порожденный процесс может завершиться раньше, чем родитель вызовет wait(2). Тогда wait(2) возвращает управление немедленно, сохранив слово состояния в своем параметре. В другом случае, подпроцесс может начать исполнение длинной и медленной программы. Тогда родитель прекращает исполнение (засыпает), пока подпроцесс не завершится. Так же, как и в первом случае, формируется слово состояния. Возвращаемое значение, идентификатор завершившегося подпроцесса, сохраняется в переменной ret.

Ожидание одного процесса - Пример

23 Родитель распечатывает значение, возвращенное wait(2). Оно должно соответствовать значению, распечатанному в строке 19.

Замечание: Если код возврата отрицательный, может возникнуть ситуация потери знака: система сохраняет в слове состояния только младший байт кода возврата, а макрос WEXITSTATUS интерпретирует его как беззнаковое число, поэтому отрицательные коды возврата будут преобразованы в положительные числа в диапазоне от 127 до 255.

Ожидание одного процесса - Пример

24-26 WIFEXITED возвращает ненулевое значение, если подпроцесс завершился нормально. Затем макросом WEXITSTATUS вычисляется код завершения подпроцесса.

28-31 WIFSIGNALED возвращает ненулевое значение, если подпроцесс был прерван сигналом. WTERMSIG используется для вычисления номера сигнала.

Пример демонстрируется следующим образом:

$ wait1 child: pid=10701 ppid=10700 exit_code=1

parent: waiting for child=10701

parent: return value=10701

child's exit status is: 1 Файл: wait1.c

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <wait.h>
#define EXIT_CODE 1

main()
{
    pid_t pid, ret;
    int status;

    if ((pid = fork()) == 0){    /* child */
    printf("child: pid=%ld ppid=%ld exit_code=%d\n",
        getpid(), getppid(), EXIT_CODE);
    exit(EXIT_CODE);
    }

    printf("parent: waiting for child=%ld\n", pid);

    ret = wait(&status);

    printf("parent: return value=%ld\n", ret);
    if (WIFEXITED(status))
    printf("child's exit status is: %d\n",
        WEXITSTATUS(status));
    else
    if (WIFSIGNALED(status))
        printf("signal is: %d\n",
        WTERMSIG(status));

    exit(0);
}
```

Ожидание нескольких процессов - Пример

В этом примере родительский процесс порождает два процесса, каждый из которых запускает команду echo(1). Затем родитель ждет завершения своих потомков, прежде чем продолжить свое исполнение. Строки 17 и 18 показывают, как использовать wait(2) в этой ситуации. wait(2) вызывается из цикла while. Он вызывается три раза. Первые два вызова ожидают завершения процессовпотомков. Последний вызов возвращает неуспех, так как некого больше ожидать. Заметьте, что код завершения потомков здесь игнорируется.

Ниже эта программа исполняется два раза. Порядок исполнения трех процессов непредсказуем.

$ wait2

this is message one

parent: waiting for children

this is message two

parent: all children terminated

$ wait2

this is message two

this is message one

parent: waiting for children

parent: all children terminated

Файл: wait2.c

```
 1 #include <sys/types.h>
 2 #include <unistd.h>
 3 #include <stdlib.h>
 4 #include <wait.h>
 5 #include <stdio.h>
 6
 7 main()
 8 {
 9     if (fork() == 0)    /* child */
10                 execl("/bin/echo", "echo", "this is",
11             "message one", (char *) 0);
12     if (fork() == 0)    /* child */
13                 execl("/bin/echo", "echo", "this is",
14             "message two", (char *) 0);
15     printf("parent: waiting for children\n");
16
17     while (wait(0) != -1)
18                 ;       /* null */
19
20     printf("parent: all children terminated\n");
21         exit(0);
22 }
```